IE540
L3. Dynamic Programming

KAIST ISE
Hayong Shin

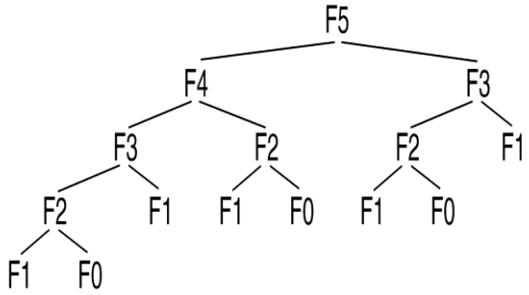
MDP vs DP vs RL

- **MDP (Markov Decision Process)**
  - Sequential decision making problem in a dynamic system
    - a.k.a. DP problem → confusing, not recommended
  - MDP : a formalism for seq. dec. making problem w/ Markov property
  - described by : state, state transition, action, reward, policy
  - Recursive formula (Bellman eq.)
- **DP (Dynamic Programming)** : algorithm for solving recursive problem
  - DP for static problems (as an algorithmic pattern)
    - recursively defined problems
    - solve smaller problem → store result → use it for bigger problem
    - eg : string matching, knapsack, shortest path, …
  - DP for MDP problem : backward induction or fixed point iteration
    - solution method for MDP : VI, PI
    - curse of dimensionality : large state space, action space
    - curse of model : unknown state transition model, reward model
- **RL (Reinforcement Learning)** ≈ Approximate DP
  - Solve DP problem approximately without knowing the model
    - Learn from experience (interaction with the environment)
    - Use functional approximation to the solution

Fibonacci with recursion

- Recursive Definition : fibo(n) =
- Recursion : starting from goal (bigger problem) → go down toward smaller problem

```
def fiboRec(n):
  if (n <= 1): return n
  return fiboRec(n-1) + fiboRec(n-2)
```



  - Lots of duplicated work : fibo(n-1) also calls fibo(n-2)
    - Number of calls grow exponentially (2^n)
    - Nearly 300,000,000 recursive calls for fibo(40)
      - fibo(40)→4s, fibo(41)→8s, fibo(42)→16s, …, , fibo(50)→4000s
- Forward evaluation of fibo(40) will require only 39 additions
  - 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, …, 102334155 → 0.001 sec
  - fibo(400) = 1.76×10^{83} → 0.0015 sec
- Lessons
  - Recursion is easy to think
    - may lead to duplicated work
  - Start from smallest problem
    - never solve the same instance of a problem
    - use it for solving a little bigger problem

Dynamic Programming for static problems

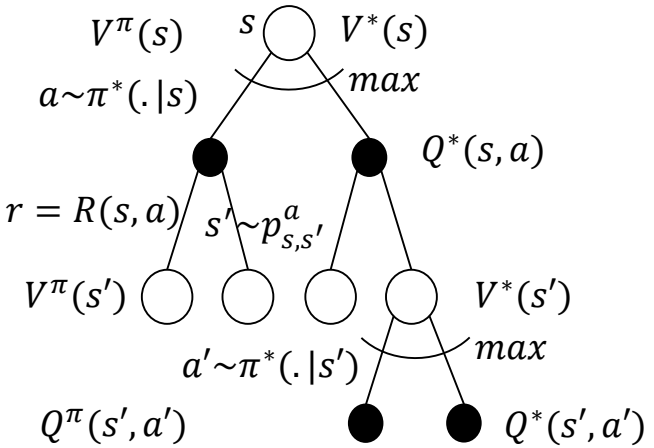
- **DP principle for recursively defined problems**
  - instead of solving the same problem again and again
  - **store and use** the result of solved (smaller) problem
- **(Bottom-up) DP : smaller → larger**
  - prepare storage space (F) for sub-problem solutions
  - start with the **smallest sub-problems**
    - base case : solution is known
    - iteration from base cases to larger problem
  - store their results (F)
  - use them to solve larger problems
  - **order : backward, forward**
    - solving order may not be obvious → Fixed point iteration

```
def fiboForward (N):
  F = [0] * (N+1)
  F[0] = 0
  F[1] = 1
  for n in range(2,N+1):
    F[n] = F[n-1] + F[n-2]
  return F[N]
```

- **Many applications**
  - Matrix chain multiplication
  - DNA sequence matching problem
  - Shortest path (Dijkstra algorithm)
  - 0-1 knapsack problem
  - Hidden Markov Model (Viterbi algorithm)

DP for MDP

- Bellman Eq. for MDP $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$
  - Model (P and R) is fully known.
  - evaluate policy $\pi(a|s)$ (= prediction)
    - Bellman expectation equation : recursive eq.
      - $V^{\pi}(s) = \mathbb{E}_{a\sim\pi}[Q^{\pi}(s,a)]$
      - $Q^{\pi}(s,a) = R(s,a) + \gamma \mathbb{E}_{s'\sim p^{a}_{s,s'}}[V^{\pi}(s')]$
  - find optimal value function (= optimal control)
    - Bellman optimality equation : recursive eq.
      - $V^{*}(s) = \max_{a} Q^{*}(s,a)$
      - $Q^{*}(s,a) = R(s,a) + \gamma \mathbb{E}_{s'\sim p^{a}_{s,s'}}[V^{*}(s')]$



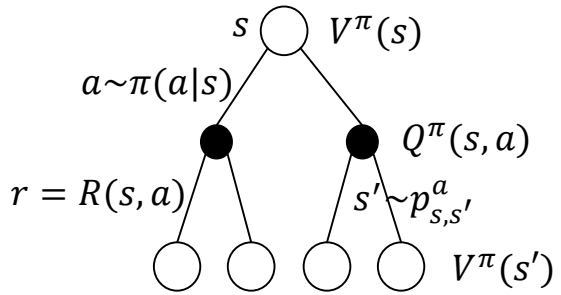
$$V^{*}(s) = \max_{a}\left[R(s,a) + \gamma \mathbb{E}_{s'\sim p^{a}_{s,s'}}[V^{*}(s')]\right]$$

$$Q^{*}(s,a) = R(s,a) + \gamma \mathbb{E}_{s'\sim p^{a}_{s,s'}}\left[\max_{a'} Q^{*}(s',a')\right]$$

- How to compute value function $V^{\pi}(s)$ or $V^{*}(s)$?
  - using DP principle : store and use smaller problem solution
  - what is smaller problem and what is bigger problem?
    - finite horizon → backward from the ending stage
    - no known horizon → ordering is not clear
      - fixed point iteration $\boldsymbol{v} = f(\boldsymbol{v})$

Iterative policy evaluation

backup diagram



- Given : model ($p^a_{s,s'}$ and $R(s,a)$), γ, $\pi(a|s)$
- Problem : find $V^\pi(s)$
- Direct solution : $\boldsymbol{V}^\pi = (\boldsymbol{I} - \gamma \boldsymbol{P}^\pi)^{-1} \boldsymbol{R}^\pi$ → O(n^3)
  - $\boldsymbol{P}^\pi = [p^\pi_{s,s'}]$, $p^\pi_{s,s'} = \sum_a \pi(a|s) p^a_{s,s'}$
  - $\boldsymbol{R}^\pi = [R^\pi(s)]$, $R^\pi(s) = \sum_a \pi(a|s) R(s,a)$
- DP solution (fixed point iteration w/ Bellman eq.)
  - initialize $V^0(s) = 0$ for $\forall s \in S$
  - for $k = 0,1,2,..$

$$V^\pi(s) = R^\pi(s) + \gamma \mathbb{E}_{\substack{a \sim \pi \\ s' \sim p^a_{s,s'}}} [V^\pi(s')]$$

$$V^{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\qquad \right] \text{ for } \forall s \in \mathcal{S}$$

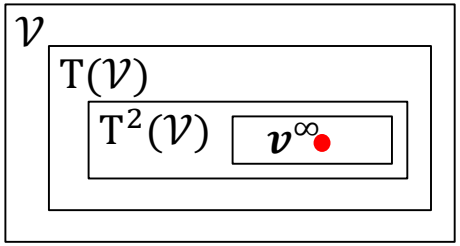
(this is called "synchronous backup")

$\boldsymbol{V}^{k+1} =$ (in matrix form)

- TAQ $\boldsymbol{V^k} \to \boldsymbol{V^\pi}$ as $k \to \infty$?
- TAQ Is DP solution better than direct solution (matrix inversion)?

Contraction mapping

- Value function space $\mathcal{V}$ of dimension $= |S|$
  - $\boldsymbol{v} = [v(s), s \in S] \in \mathcal{V}$ means a vector of state values
  - distance metric : $\infty - \text{norm}$ i.e. $\|\boldsymbol{u} - \boldsymbol{v}\|_\infty = \max_{s \in S} |u(s) - v(s)|$
- Contraction mapping
  - Let T be a mapping defined on $\mathcal{V}$
  - Assume that $\mathcal{V}$ is closed under T (i.e. $T(\boldsymbol{v}) \in \mathcal{V}$ for $\forall \boldsymbol{v} \in \mathcal{V}$)
  - T is γ −contraction if $\|T(\boldsymbol{u}) - T(\boldsymbol{v})\|_\infty \leq \gamma \|\boldsymbol{u} - \boldsymbol{v}\|_\infty$ for $\forall u, v \in \mathcal{V}$
    - distance gets smaller by the factor of γ at least
  - Let T^k denote k-times application of T
    - $T^k(v) = T\left(T\left(\ldots T(v)\right)\right)$



- Contraction mapping theorem
  - if T is a γ −contraction mapping with $\gamma < 1$,
    - for $\forall \boldsymbol{v} \in \mathcal{V}$, $T^k(\boldsymbol{v})$ converges to a unique fixed point $\boldsymbol{v}^\infty$ as $k \to \infty$
    - at a linear convergence rate of γ

$$\lim_{k \to \infty} \frac{\left|T^{k+1}(\boldsymbol{v}) - \boldsymbol{v}^\infty\right|}{|T^k(\boldsymbol{v}) - \boldsymbol{v}^\infty|^q} = \gamma\,, \qquad \begin{matrix} q\text{: convergence order} \\ (q = 1 : \text{linear conv.})' \end{matrix} \qquad \gamma\text{: convergence rate}$$

Does policy evaluation converge?

- Define Bellman expectation backup operator T^{π}

$$T^{\pi}(\boldsymbol{v}) = \boldsymbol{R}^{\pi} + \gamma \boldsymbol{P}^{\pi} \boldsymbol{v}$$

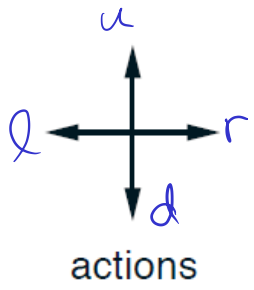
- T^{π} is a γ −contraction mapping

$$\begin{aligned} \|T^{\pi}(\boldsymbol{u}) - T^{\pi}(\boldsymbol{v})\|_{\infty} &= \|\boldsymbol{R}^{\pi} + \gamma \boldsymbol{P}^{\pi} \boldsymbol{u} \; - \boldsymbol{R}^{\pi} - \gamma \boldsymbol{P}^{\pi} \boldsymbol{v} \; \|_{\infty} \\ &= \|\gamma \boldsymbol{P}^{\pi} (\boldsymbol{u} - \boldsymbol{v}) \; \|_{\infty} \\ &\leq \|\gamma \boldsymbol{P}^{\pi} \|\boldsymbol{u} - \boldsymbol{v}\|_{\infty} \; \|_{\infty} \\ &\leq \gamma \|\boldsymbol{u} - \boldsymbol{v}\|_{\infty} \end{aligned}$$

- Hence, $\boldsymbol{V}^{k+1} = T^{\pi}(\boldsymbol{V}^{k})$ converges to $\boldsymbol{V}^{\pi}$

- Is DP ($\boldsymbol{V}^{k+1} = T^{\pi}(\boldsymbol{V}^{k})$) faster than direct method (matrix inv.)?
  - Not necessarily. Possibly it can be much slower.
  - What if state space very large? → Matrix inv. cannot be obtained.
    - Iterate DP solution as many times as we can afford.
      - solution can be close enough.

Example : Small Gridworld (RLAI Ex4.1)



T	1	2	3
4	5	6	7
8	9	10	11
12	13	14	T

$S = \{1, \cdots, 14, T\}$ $\quad \gamma = 1$

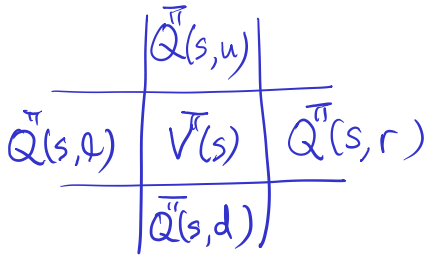
$A = \{l, r, u, d\}$

deterministic state transition

$S' = f(s,a)$ [move in a direction
stay if hit border

$$R(s,a) = \begin{cases} 0 & \text{if } s = T \\ -1 & \text{o.w.} \end{cases}$$

- Ex. Let π be equiprobable random policy i.e. $\pi(a|s) = 1/4$ for $\forall s, a$
  - $Q^{\pi}(11, \mathrm{d}) =$
  - $Q^{\pi}(7, \mathrm{d}) =$



$$Q^\pi(s,u) = -1 + V^\pi(s')$$

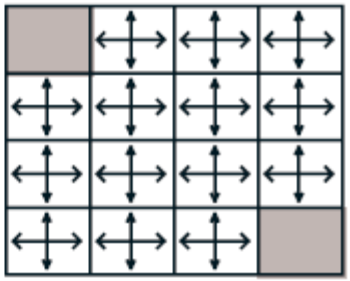
$$\arg\max_{a \in A} Q^\pi(s,a)$$

v_k for the random policy

greedy policy w.r.t. v_k

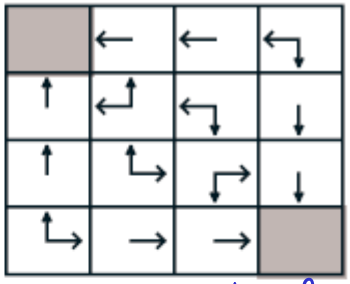
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k=0$



0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

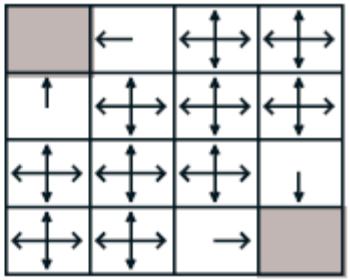
$k=3$



all optimal policy

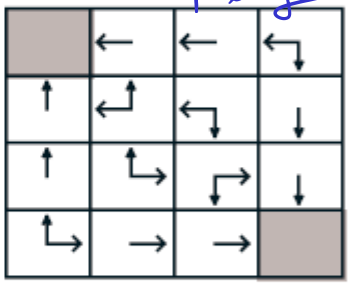
0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k=1$



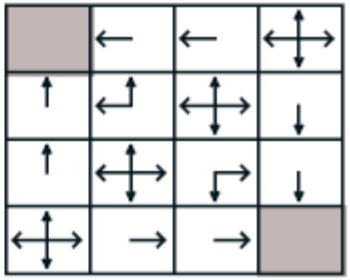
0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k=10$



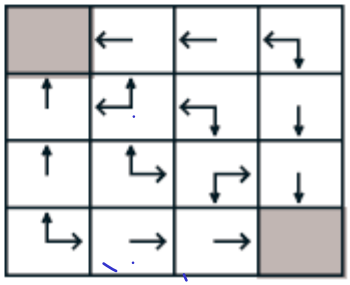
0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k=2$

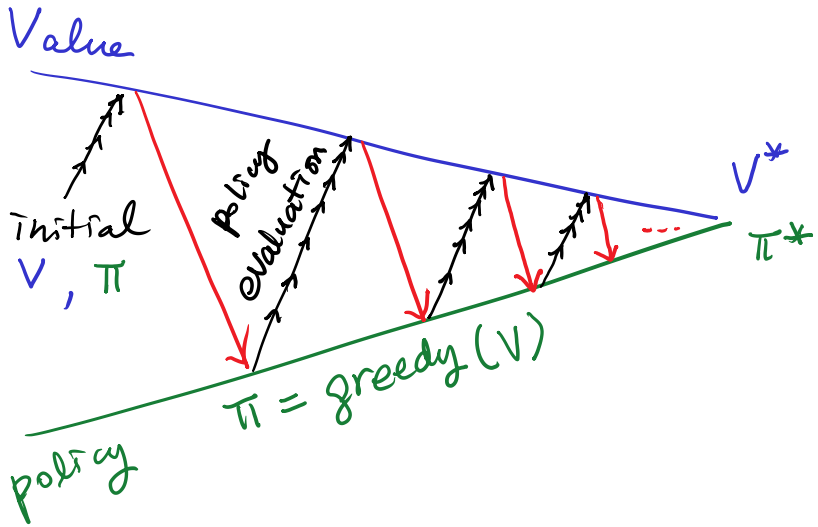


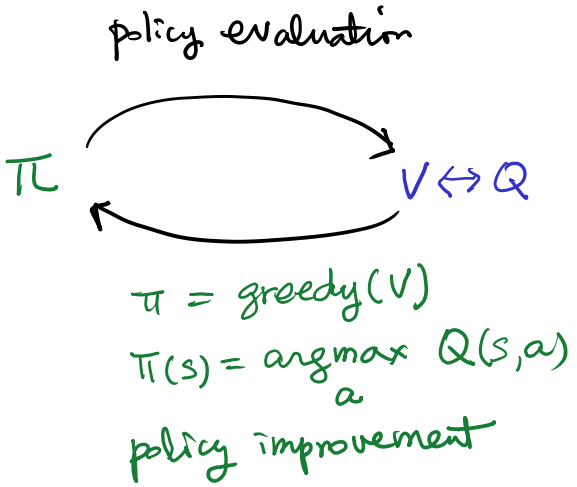
0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

$k=\infty$



Policy Iteration





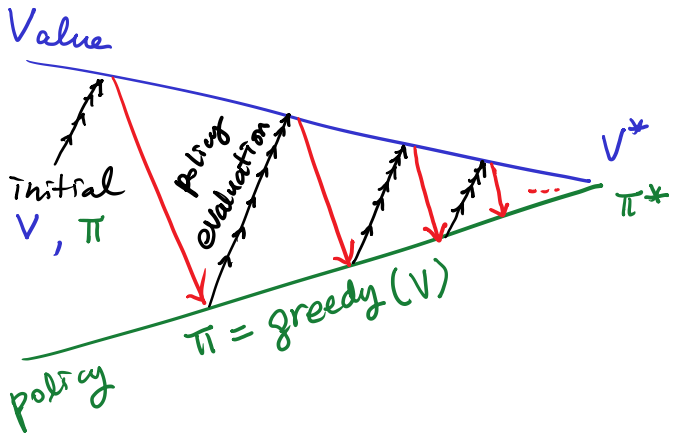
- Policy iteration
  - repeat
    - Policy evaluation : Estimates $V^{\pi}(s)$
      - iterative policy evaluation
    - Policy improvement : Generate $\pi' \geqslant \pi$
      - greedy policy improvement $\pi' = \text{greedy}(V^{\pi})$
        - » $\pi'(s) = \underset{a}{\text{argmax}}\, Q^{\pi}(s,a)$

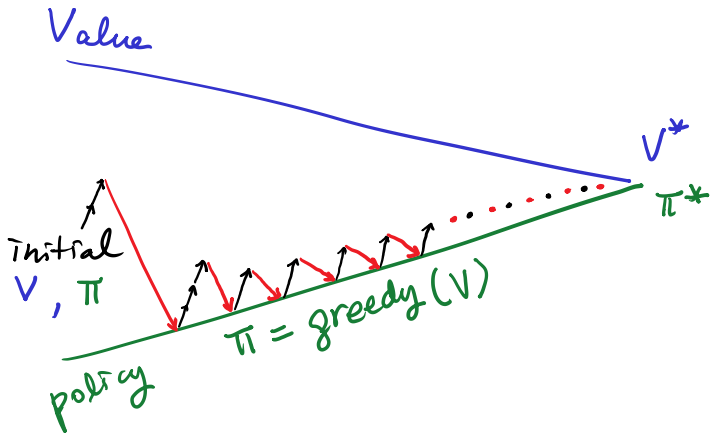
How to improve a policy π in PI

- Using state-value function $V^{\pi}(s)$
  - Evaluate the policy π
    $V^{\pi}(s) = \mathbb{E}[G_t|S_t = s] = \mathbb{E}[R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots |S_t = s]$
  - Improve the policy by choosing greedy action w.r.t. V^{π}
    - $\pi' = \text{greedy}(V^{\pi})$: for each state s, find the best action from s based on V^{π}
    - Using action-value function $Q^{\pi}(s,a) = R(s,a) + \gamma \mathbb{E}_{s' \sim p^a_{s,s'}}[V^{\pi}(s')]$
    - $\pi'(s) = \underset{a}{\text{argmax}}\, Q^{\pi}(s,a)$: greedy action w.r.t. Q^{π}
  - This step gives improved policy : $V^{\pi}(s) \leq V^{\pi'}(s)$ i.e. $\pi \preccurlyeq \pi'$
    - $Q^{\pi}\big(s, \pi'(s)\big) = \max_a Q^{\pi}(s,a) \geq Q^{\pi}\big(s, \pi(s)\big) = V^{\pi}(s)$
    - $V^{\pi}(s) \leq Q^{\pi}\big(s, \pi'(s)\big) = \mathbb{E}_{\pi'}[R_t + \gamma V^{\pi}(S_{t+1})|S_t = s]$
      $\leq \mathbb{E}_{\pi'}\big[R_t + \gamma Q^{\pi}\big(S_{t+1}, \pi'(S_{t+1})\big)|S_t = s\big]$
      $\leq \mathbb{E}_{\pi'}\big[R_t + \gamma R_{t+1} + \gamma^2 Q^{\pi}\big(S_{t+2}, \pi'(S_{t+2})\big)|S_t = s\big]$
      $\leq \mathbb{E}_{\pi'}[R_t + \gamma R_{t+1} + \cdots |S_t = s] = V^{\pi'}(s)$
  - If improvements stop : optimal
    - $Q^{\pi}\big(s, \pi'(s)\big) = \max_a Q^{\pi}(s,a) = Q^{\pi}\big(s, \pi(s)\big) = V^{\pi}(s)$
    - V^{π} satisfies the Bellman optimality eq. : $V^{\pi}(s) = \max_a Q^{\pi}(s,a)$
    - V^{π} is optimal ($V^{\pi}(s) = V^*(s)$), so is π ($\pi = \pi^*$)

Generalized Policy Iteration

- How many iteration of policy evaluation?
  - ϵ-convergence to V^{π} ?
    - stop if max change in $V^{\pi}(s) < \epsilon$. how to set ϵ ?
  - fixed k iteration ?
    - how to set k ?
  - stop after $k = 1$ iteration → update policy at every iteration of evaluation
    - equivalent to **value iteration**
- Generalized Policy Iteration
  - any policy evaluation, which estimates V^{π}
  - any policy improvement, which generates $\pi' \geqslant \pi$





PI w/ $k = 1$

- PI w/ $k = 1$ (deterministic policy)
  - initialize V
  - repeat
    - Policy improvement

$$\pi(s) = \underset{a}{\operatorname{argmax}} \left[R(s,a) + \gamma \sum_{s'} p_{s,s'}^{a} V(s') \right]$$

    - Policy evaluation : Estimates $V^{\pi}(s)$

$$V^{new}(s) = R(s,\pi(s)) + \gamma \sum_{s'} p_{s,s'}^{\pi(s)} V(s') \quad \text{for } \forall s \in \mathcal{S}$$

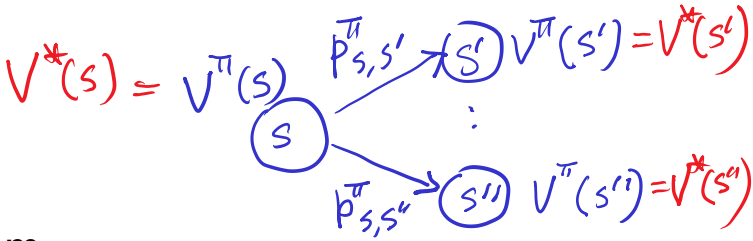
$$V \leftarrow V^{new}$$

- PI w/ $k = 1$ (merge two steps)
  - initialize V
  - repeat

$$V^{new}(s) = \max_{a} \left[R(s,a) + \gamma \sum_{s'} p_{s,s'}^{a} V(s') \right] \text{for } \forall s \in \mathcal{S}$$

$$V \leftarrow V^{new}$$

Value Iteration

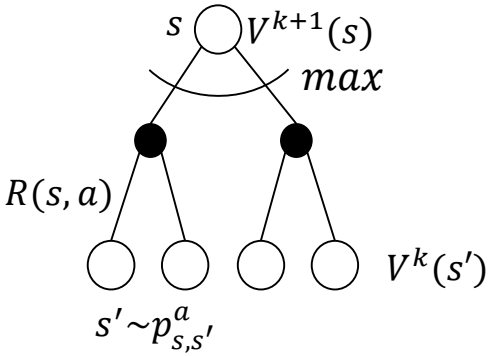


- Theorem (Principle of Optimality)
  - $V^\pi(s)$ is optimal (i.e. $V^\pi(s) = V^*(s)$)

  $\Leftrightarrow V^\pi(s')$ is optimal for any state s' reachable from s

  (i.e. $V^\pi(s') = V^*(s')$ for $\forall s'$ with $p^\pi_{s,s'} = \sum_a \pi(a|s) p^a_{s,s'} > 0$
- Idea to get $V^*(s)$
  - if we know the solution to subproblems $V^*(s')$
  - use them to get $V^*(s) \leftarrow \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} p^a_{s,s'} V^*(s') \right]$
  - start from terminal state → work backwards
  - in many case, order is not clear (or loopy) → use fixed point iteration
- **VI** (fixed point iteration w/ Bellman opt. eq.)
  - initialize $V^0(s) = 0$ for $\forall s \in S$
  - for $k = 0,1,2,..$

$$V^{k+1}(s) = \max_{a \in \mathcal{A}} \left[\quad \right] \text{ for } \forall s \in \mathcal{S}$$



$$\boldsymbol{V}^{k+1} = \max_{a \in \mathcal{A}} [\quad] \quad \text{(in matrix form)}$$

Example : Shortest path

- $\mathcal{S} = \{0,1,..,15\}$, state 0 (marked as 'g') is the terminal state
- $\mathcal{A} = \{l, r, u, d\}$, $\gamma = 1$
- deterministic transition
- reward = -1 per each step, reward = 0 at the goal state

g			

Problem

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

V_1

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

V_2

0	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	-2

V_3

0	-1	-2	-3
-1	-2	-3	-3
-2	-3	-3	-3
-3	-3	-3	-3

V_4

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-4
-3	-4	-4	-4

V_5

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-5

V_6

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-6

V_7

Does value iteration converge?

- Define Bellman optimality backup operator T^*

$$T^*(\boldsymbol{v}) = \max_{a \in \mathcal{A}} \left[\boldsymbol{R}^{\boldsymbol{a}} + \gamma \boldsymbol{P}^{\boldsymbol{a}} \boldsymbol{v}\right]$$

- T^* is a γ −contraction mapping

$$\|T^*(\boldsymbol{u}) - T^*(\boldsymbol{v})\|_\infty \leq \gamma \|\boldsymbol{u} - \boldsymbol{v}\|_\infty$$

(proof is similar as previous)

- Hence, $\boldsymbol{V}^{k+1} = T^*\left(\boldsymbol{V}^k\right)$ converges to $\boldsymbol{V}^*$

- TAQ. Direct method (matrix inv.) for solving Bellman optimality eq.?

- TAQ. Is VI faster than PI?

DP Algorithms for MDP : Policy Eval, PI, VI

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + Greedy Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

- state value function based : $V^{\pi}(s)$ or $V^{*}(s)$
  - complexity : $O(|\mathcal{S}|^2|\mathcal{A}|)$ per iteration
- can be applied to action value functions : $Q^{\pi}(s,a)$ or $Q^{*}(s,a)$
  - complexity : $O(|\mathcal{S}|^2|\mathcal{A}|^2)$ per iteration
- Now we have algorithms for solving MDP. Are we done?
  - Curse of
    - $|\mathcal{S}|, |\mathcal{A}|$ are usually prohibitively large (or infinite) even for a single iteration
  - Curse of
    - state transition and reward model is unknown in most cases

Asynchronous DP

- Synchronous DP backs-up all states together (use V and V^{new})

$$V^{new}(s) \leftarrow \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} p^{a}_{s,s'} V(s') \right] \text{for } \forall s \in \mathcal{S}$$

$$V \leftarrow V^{new}$$

- Asynchronous DP backs-up each state individually, in any order
  - significantly reduce computation (faster convergence)
  - convergence is still guaranteed
  - In-place DP, prioritized sweep, real-time DP
    - can be applied to PI (policy eval.) and VI

- In-place DP : keep only single copy of V

$$V(s) \leftarrow \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} p^{a}_{s,s'} V(s') \right] \text{for } \forall s \in \mathcal{S}$$

  - space reduction, simpler code
  - a little faster convergence $\because$ new information in $V(s)$ is used promptly

Asynchronous DP – Prioritized sweeping

- optimal value function $V^*(s)$ satisfies Bellman optimality eq.

$$V^*(s) = \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} p^a_{s,s'} V^*(s') \right]$$

- *Bellman error* (a.k.a. Temporal Difference error) of $V(s)$

$$BE(s) = \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} p^a_{s,s'} V(s') \right] - V(s)$$

  - $|BE(s)|$ is big
    → $V(s)$ is far from the correct value
    → $V(s)$ needs to be corrected before it is backed up for other state

- Prioritized sweeping
  - compute and store $|BE(s)|$ for $\forall s \in \mathcal{S}$
  - repeat
    - backup state with biggest $|BE(s)|$
    - update $|BE(s')|$ of state s' affected by this backup
  - TAQ. Data structure for efficiently storing $|BE(s)|$?

Real-time DP

- VI (or similarly in PI) backs up every state in each iteration

$$V(s) = \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} p^a_{s,s'} V(s') \right] \text{ for } \forall s \in \mathcal{S}$$

  - Is it necessary to obtain better value of $V(s)$ for state that will never be visited?
- Real-time DP
  - note that DP implies the model is available
  - idea : update states that are relevant to the agent only
  - how : backup states while following sample trajectory $(S_0, A_0, R_0, S_1, A_1, R_1, \dots, S_T)$
  - at each time step t, backup the state $s = S_t$ only

$$V(s) \leftarrow \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} p^a_{s,s'} V(s') \right]$$

  - There are many questions unanswered yet
    - How to select action A_t?
    - How to utilize sample reward R_t?
    - …
  - consider Real-time DP as a bridge from DP to RL

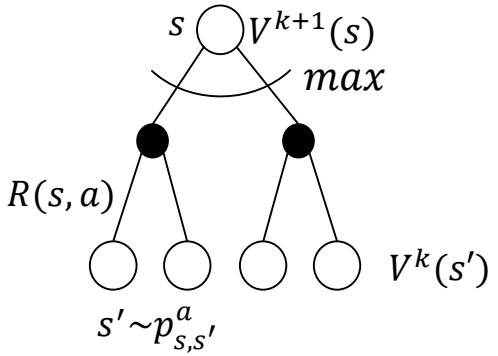
Approximate Dynamic Programming (ADP)

- ADP : use function approximation to break curse of dimensionality
  - model ($R(s,a)$ and $p_{s,s'}^{a}$) is known
  - $|\mathcal{S}|$ gets too big
    - can be infinite if state is continuous
  - Lookup table for $V(s)$: explicitly storing $V(s)$ for each s
    - too big to compute or store
  - Function approximator $\hat{V}(s;w)$ instead $V(s)$
- Fitted VI
  - initialize w_0
  - for $k = 0,1,2,3,\ldots$
    - for $i = 1,\ldots,n$ (n : sample size per iteration)
      - sample $s_i \in \mathcal{S}$
      - estimate target value using Bellman eq.

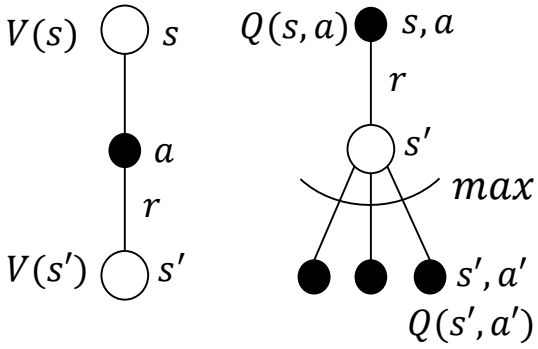
$$v_i = \max_{a\in\mathcal{A}} \left[R(s,a) + \gamma \sum_{s'\in\mathcal{S}} p_{s,s'}^{a} \hat{V}(s;w_k) \right]$$

      - add $\langle s_i, v_i \rangle$ into $\mathcal{D}_{k+1}$
    - fit $\hat{V}(s;w_{k+1})$ to $\mathcal{D}_{k+1}$: $w_{k+1} \leftarrow \underset{w}{\operatorname{argmin}} \sum_{i\in\mathcal{D}_{k+1}} \left(v_i - \hat{V}(s_i;w)\right)^2$

Full-with backup vs Sample backup



- Full-width backup
  - update $V(s)$ using all possible successors $V(s')$
  - DP uses full-width backup
    - effective for medium-sized problems
      - up to millions of states
    - for larger problems, DP suffers Bellman's curse of dimensionality
      - let $s = (x_1, x_2, \ldots, x_n)$, where x_i is a binary variable
      - state space grows exponentially: $|\mathcal{S}| = 2^n$
      - even one backup can be too expensive



- Sample backup
  - from Lecture 4, *sample backup* will be considered
  - use sample segment (s, a, r, s')
    - obtained from the interaction w/ environment
  - advantage
    - model-free : no knowledge of model is assumed
    - break curse of dim.